DESIGN AND DEVELOPMENT OF AN AGENTIC AI-DRIVEN PREDICTIVE MAINTENANCE ANALYTICS AND WORKFLOW AUTOMATION SYSTEM USING MICROSOFT COPILOT STUDIO

by
SARAH KHALILAH BINTI MOHD FAUZI
22005955

Interim Report submitted in partial fulfilment of the requirements for the
Bachelor of Information Technology (Hons)
TFB3012 - Final Year Project I

Universiti Teknologi PETRONAS
32610 Seri Iskandar
Perak Darul Ridzuan

CERTIFICATION OF APPROVAL

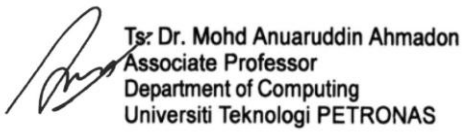
DESIGN AND DEVELOPMENT OF AN AGENTIC AI-DRIVEN PREDICTIVE MAINTENANCE ANALYTICS AND WORKFLOW AUTOMATION SYSTEM USING MICROSOFT COPILOT STUDIO

by

SARAH KHALILAH BINTI MOHD FAUZI
22005955

Interim report submitted in partial fulfilment of the requirements for the
Bachelor of Information Technology (Hons)

Approved by,



AP Ts Dr Mohd Anuaruddin bin Ahmadon

CERTIFICATION OF ORIGINALITY

This is to certify that I am responsible for the work submitted in this project, that the original work is my own except as specified in the references and acknowledgements, and that the original work contained herein has not been undertaken or done by unspecified sources or persons.



_____

SARAH KHALILAH BINTI MOHD FAUZI

ABSTRACT

Condition monitoring and predictive maintenance in the context of smart manufacturing has become increasingly significant due to increased integration of the production processes, where failure in a hydraulic circuit may trigger other faults, causing additional costs and equipment downtimes. Current solutions for monitoring the conditions of the equipment may be divided into two groups: either dashboard analytics that require manual extraction, comparison and interpretation of visual data, or machine learning models for predictions without any action. None of these solutions alone can close the cycle from detection of a problem to acting upon it.

In this project, we address this problem by creating an integrated solution for predictive maintenance, that will include not only the detection of failures, but also dashboard analytics, conversation agent interaction and automation of the workflow. These four solutions are implemented on the same data layer, using the Microsoft Power Platform technology. Contrary to previous solutions that were organized as separate steps in the pipeline, in our solution all four modules work in a hub and spoke architecture, where Power BI, Microsoft Copilot Studio and Power Automate are the parallel consumers of the data layer.

The system is built and evaluated using the UCI Condition Monitoring of Hydraulic Systems dataset, a real experimental dataset comprising 2,205 load cycles collected from 17 sensors on a physical hydraulic test rig, chosen specifically over synthetic or simulated alternatives to ensure the validity of the resulting model. By the end of Final Year Project I, the raw sensor data has been processed through a feature extraction pipeline, an initial failure-prediction model has been trained and evaluated against a rule-based baseline, an analytics dashboard has been constructed in Power BI, and the automated alerting and work-order workflow has been implemented such that the system can autonomously convert a high-risk prediction into a maintenance action without human intervention.

Feature extraction, model training, and evaluation have been completed using the entire dataset. For the four components monitored (cooler, valve, pump, accumulator), the top-performing model achieved a cross-validated macro-F1 score ranging from 0.982 to 0.999 and a macro PR-AUC between 0.997 and 1.000. This performance is contrasted with a rule-based baseline (a single-split decision stump) which yielded a macro-F1 of only 0.42-0.56. Notably, for the cooler component, the baseline model achieved 0% recall on the Critical

class. The improvements over the baseline were statistically significant across all components (paired t-test, $p < 0.0001$). Analysis of SHAP attributions revealed that the maximum cooling power (CP_max) recorded within a cycle was the most influential feature for predicting cooler condition, which aligns with the component's physical principles.

Final Year Project II will extend this foundation with the conversational agent layer in Microsoft Copilot Studio, full integration testing across all four Power Platform components, and a controlled user study comparing the integrated system against a dashboard-only baseline on decision-making effectiveness and perceived workload.

TABLE OF CONTENTS

ABSTRACT iv
CHAPTER 1: INTRODUCTION 1
1.1 Background **1**
1.2 Problem Statement……..**2**
1.3 Objectives……**3**
1.4 Scope of Study……..**4**
CHAPTER 2: LITERATURE REVIEW / THEORY 5
2.1 Condition-Based Maintenance and Predictive Maintenance Foundations……...**5**
2.2 Machine Learning for Fault Prediction, Including Hydraulic-Specific Work……..**5**
2.3 Agentic AI in Maintenance Contexts……..……**5**
2.4 AI-Based Decision Support and Actionable Maintenance Systems……...**6**
2.5 Conversational Interfaces versus Dashboard-Based Decision Support……**6**
2.6 Industry Practice: Low-Code Agent Platforms……..…**6**
2.7 Evaluation Instruments for Human-Facing Systems……..**7**
2.8 Research Gap……..**7**
CHAPTER 3: METHODOLOGY……...**8**
3.1 CRISP-DM Process Model……**8**
3.2 System Architecture…..……**9**
3.2.1 End-to-End Flow: Autonomous Path (FYP1, Must Actually Run) ……**10**
3.2.2 End-to-End Flow: Conversational Path (FYP2, Design Only)……..……....**12**
3.2.3 Known Limitations to Address Before Submission……**13**
3.3 Dataset and Data Preparation ..... Error! Bookmark not defined.**4**
3.4 Tools and Platform **95**
CHAPTER 4: RESULTS AND DISCUSSION ..... **Error! Bookmark not defined.6**
4.1 Data Preparation Results ..... Error! Bookmark not defined.**6**
4.2 Machine Learning Model Results ..... Error! Bookmark not defined.**6**
4.3 Dashboard Development **168**
4.4 Progress Against Objectives **189**
CHAPTER 5: CONCLUSION AND FUTURE WORKS..……..……**20**
REFERENCES……**23**

CHAPTER 1: INTRODUCTION

1.1 Background

A hydraulic system is composed of interdependent components, including pumps for flow generation, valves for directional and pressure control, coolers for managing operational thermal loads, and accumulators for energy storage and pressure fluctuation mitigation. The inherent physical and functional coupling among these components means that component degradation typically affects multiple parts of the system. For instance, a less efficient cooler can lead to increased system temperatures, which subsequently elevates thermal and mechanical stress on valves and pumps, thereby accelerating wear. Consequently, an undetected minor fault in such a coupled circuit can escalate into a more significant and costly failure.

The cost associated with propagating a failure is primarily time dependent. Identifying a degrading component earlier generally leads to less expensive and less disruptive interventions. This principle underpins condition-based and predictive maintenance strategies, which leverage sensor data to assess equipment health directly, rather than adhering to predetermined maintenance schedules.

However, accurate prediction alone is insufficient. A predictive model that identifies a degrading valve, for instance, is only valuable if this information is communicated promptly and in an actionable format to the appropriate personnel. This necessitates a system that extends beyond prediction to include automated alerting, workflow initiation, and an accessible interface. Such an interface would allow both technical and non-technical staff to inquire about equipment status.

This project implements such a system using the Microsoft Power Platform. It integrates Power BI for data analysis, a machine learning model for failure prediction, Microsoft Copilot Studio for conversational interfaces, and Power Automate for workflow management. These components are unified by a common data layer, functioning as an integrated system rather than four disparate tools.

1.2 Problem Statement

Traditional maintenance practice in the context of manufacturing environments is mostly reactive or schedule-based rather than condition based, which implies that the former is executed either after the occurrence of a failure or at regular time intervals regardless of the actual state of the equipment. This approach, however, introduces a delay in the response to potential malfunctions, thus eliminating the possibility of realizing the economic and safety benefits of condition based maintenance (Jardine, Lin, & Banjevic, 2006; Carvalho et al., 2019).

Since hydraulic components are closely related to one another both mechanically and thermally, a malfunction of one part disrupts the entire circuit and aggravates the consequences of the failure, which in turn increases the costs associated with maintenance. For this reason, it is essential to close the loop between prediction and action, not only detecting the malfunction but also addressing it accordingly (Christou, Kefalakis, Zalonis, Soldatos, & Bröchler, 2020).

The existing dashboards, predictive analytic tools, and alerting mechanisms are typically disjointed, using different interfaces and not sharing information. While prior literature has built integrated analytic pipelines, these works do not provide examples of a technically realized closed loop system where predictions, interactions, and responses are all built on a shared information layer. (Bousdekis, Magoutas, Apostolou, & Mentzas, 2018; Jiang & Hu, 2025).

Last but not least, while intuitive, analytics dashboards still require a cognitive effort from the user to piece together the conclusions and track down the root causes. This burden typically falls on the maintenance technicians, who are the lowest link in the managerial hierarchy, but may not have a technical background to grasp the analytics and their implications. In a direct comparison of an LLM-based conversational agent to a dashboard in the industrial analytics context, the language model was shown to reduce the perceived workload and be faster at lower cognitive complexity tasks, but outperformed by the dashboard as the complexity increases (Figliè et al., 2026). This positions such conversational agents as a potential addition to the analytic pipeline, helping less technically

inclined workers navigate the dashboard and perform simpler queries, while being supplemented for more involved tasks.

1.3 Objectives

The general objective of this project is to design, develop and evaluate an integrated predictive maintenance system in which machine learning based failure prediction, conversational agent interaction, and automated maintenance workflows operate as a single closed-loop pipeline on the Microsoft Power Platform.

The specific objectives are as follows:

- O1 - To develop a machine learning-based failure prediction model using real, multi-sensor hydraulic system data, addressing class imbalance and benchmarked against a rule-based baseline.
- O2 - To design and implement an integrated system architecture that unifies Power BI, Microsoft Copilot Studio and Power Automate through a common data layer.
- O3 - To evaluate the integrated system against a dashboard-only baseline in terms of predictive performance, decision-task effectiveness, and automation response efficiency.

Table 1.1: Mapping of Objectives to FYP1 and FYP2 Deliverables

Objective	FYP1 Deliverable	FYP2 Deliverable
O1 - Predict	Feature extraction pipeline; trained model evaluated against rule-based baseline using stratified cross-validation	Model refinement; SHAP-based interpretability feeding agent explanations
O2 - Integrate	Hub-and-spoke architecture designed and implemented; Power BI dashboard; four Power Automate flows including autonomous alert-to-work-order path	Full Copilot Studio agent build, published to a Teams channel; end-to-end integration testing
O3 - Evaluate	Model evaluation protocol executed (Part A); user study protocol designed (Parts B & C)	User study executed (N=20, within-subjects); system performance and agent quality evaluation

1.4 Scope of Study

In Scope:

- Four hydraulic components: cooler, valve, pump, and accumulator condition.
- The UCI Condition Monitoring of Hydraulic Systems dataset (real, experimental, 2,205 cycles, 17 sensors).
- Batch mode failure prediction (offline processing, not real-time streaming).
- Integration across Power BI, Microsoft Copilot Studio, Power Automate, and a shared Dataverse/SharePoint data layer.

Out of Scope:

- Real-time/streaming sensor ingestion.
- Equipment types beyond the hydraulic test-rig components listed above (e.g. CNC spindles, gearboxes).
- Full production deployment on live factory infrastructure.
- Execution of the FYP2 user study, which is designed in this report but not yet conducted.

It should be noted that batch-mode scoring is an explicitly acknowledged limitation rather than an oversight: FYP1 implements offline, scheduled prediction, while a live scoring endpoint is planned as an FYP2 extension.

CHAPTER 2: LITERATURE REVIEW / THEORY

This chapter aims to cover ten sources retrieved from seven topics, including the principles of condition-based maintenance, machine learning for predictive maintenance with the focus on hydraulics, agentic artificial intelligence for maintenance, AI-based decision-making for maintenance, the user interface (dialogue versus dashboard), industrial practice and low-code agents for maintenance, and evaluation methods for human-centered technologies. All the sources were screened in terms of the publisher's information to ensure that they were not included in the previous study, which had an issue with citation integrity.

2.1 Condition-Based Maintenance and Predictive Maintenance Foundations

Condition based maintenance (CBM) relies on data gathered from the equipment to inform maintenance scheduling as opposed to fixed time intervals. Jardine et al., 2006 provide 'a review on condition-based maintenance' and 'the basic acquisition-extraction-diagnosis-prognosis framework' which underpins this project (Jardine, Lin and Banjevic 2006). Carvalho et al., 2019 provides a 'systematic review of machine learning-based approaches for predictive maintenance', classifying predictive maintenance based on the type of problem addressed primarily classification and regression problems. A more recent publication provides a comparison between 'knowledge-based, physics-based and data driven' approaches noting 'hybridisation' of different methodologies are preferable (El Hammoumi et al., 2025).

2.2 Machine Learning for Fault Prediction, Including Hydraulic-Specific Work

Class imbalance is often an issue in ML-based fault prediction since failures are rare events compared to the normal operation of an object. Jamshidi, Kim and Arif (2025) note that imbalanced data and the high-dimensionality of features are some of the persistent issues in the domain, which corresponds to the imbalance treatment discussed in Chapter 5. In particular, hydraulic systems benefit from implementing random-forest recursive-feature-elimination, combined with XGBoost, for multi-class fault mode recognition, which reduces feature redundancy (Song et al., 2023). Both the extraction method and the utilization of XGBoost are included in the current project's scope.

2.3 Agentic AI in Maintenance Contexts

Recent research introduces agentic architectures which involve AI systems performing diagnostic and responsive reasoning beyond mere predictions. Jiang & Hu (2025) suggest a predictive maintenance framework organized around multiple autonomous agents (detection, classification, diagnosis, and simulation agents) and position PdM maintenance as the multi-agent coordination problem. Di Maggio (2025) provides an overview of the emergent literature on large language models and autonomous agents for PdM, arguing that the value of autonomous agents for the time being is focused in their informational and decision-support capabilities, with more autonomous agents requiring better coordination and governance, which is reflected in this project's limitation 2, restricting the autonomous work agent to one type of action (FL_CreateWorkOrder). The two paths of creating work orders through autonomous work agent and through dialog are similar to the two options discussed in both works.

2.4 AI-Based Decision Support and Actionable Maintenance Systems

Soori, Jough, Dastres and Arezoo (2024) examine the application of AI-enabled decision support systems in the setting of the fourth industrial revolution, highlighting the importance of data quality, integration complexity, and the need for real-time decision making as barriers to adoption, all of which are resolved by the proposed data layer in the hub-and-spoke configuration (Section 3.2). Meanwhile, Christou, Kefalakis, Zalonis, Soldatos and Bröchler (2020) describe an end-to-end industrial internet of things platform that was deployed across two factories, enabling condition monitoring and reducing downtime, while also illustrating the limitations of predictive maintenance in the absence of a closed-loop feedback mechanism, thus addressing O2.

2.5 Conversational Interfaces versus Dashboard-Based Decision Support

The primary means of presenting PdM information is still through dashboards, but they put a real burden of interpretation on the user. Figliè et al. (2026) compare directly an LLM-based agent to a dashboard (134 industrial decision-makers, tasks of increasing complexity), finding that the agent reduced perceived workload and was faster on simpler tasks, but showed no consistent accuracy advantage, narrowing as complexity increased. This supports

a conversational layer as an accessibility improvement for simple, high-frequency queries, and confirms that dashboards should remain primary for complex diagnostics.

2.6 Industry Practice: Low-Code Agent Platforms

Microsoft's own Scenario Library documents a maintenance-prediction agent pattern on Copilot Studio sensor ingestion, degradation detection, forecasting, and automated notification (Microsoft, Maintenance Prediction Agent). This is not peer-reviewed, but it does confirm the architectural direction taken here, Power Platform components coming together around a shared data layer as an industrial pattern documented by Microsoft, not an idiosyncratic choice.

2.7 Evaluation Instruments for Human-Facing Systems

The System Usability Scale (Brooke, 1996) provides a standardised ten-item usability measure, and the NASA Task Load Index (Hart & Staveland, 1988) measures workload across six dimensions. Both are used by Figliè et al. (2026) in the study discussed in Section 2.5, and both are adopted in this project's evaluation methodology (Chapter 5).

2.8 Research Gap

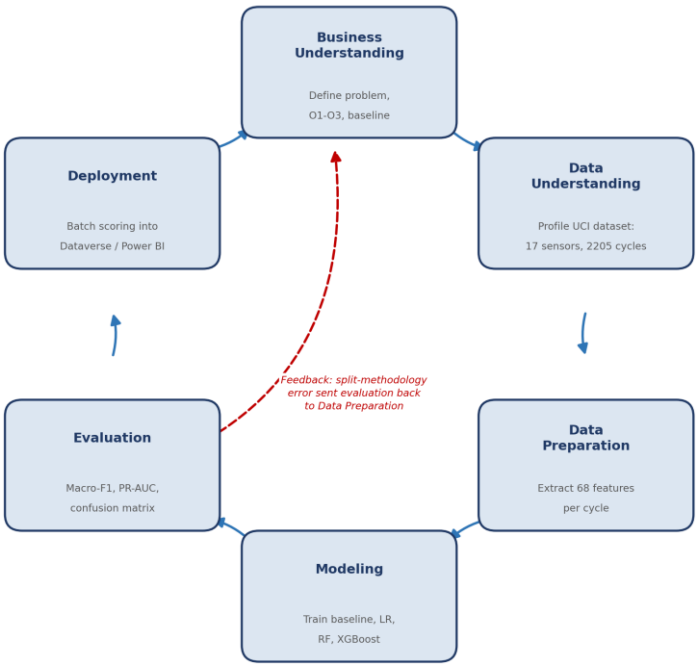
Condition-based maintenance, hydraulic fault classification, agentic AI coordination, actionable decision support, and conversational-versus-dashboard interaction have each been studied substantially, but largely in isolation. Christou et al. (2020) demonstrate a closed-loop platform without a conversational layer; Jiang and Hu (2025) and Di Maggio (2025) describe agentic patterns not evaluated against a dashboard baseline on real hydraulic data; Figliè et al. (2026) compare conversational and dashboard interaction, but in a general context, not one built on a validated hydraulic model. No source combines a real dataset, validated prediction, an autonomous alert-to-action workflow, and a conversational layer, evaluated end-to-end against both baselines. This project's three objectives map onto closing that gap: O1 establishes a credible predictive foundation, O2 architects the closed-loop integration prior agentic-AI work has not implemented on hydraulic data, and O3 evaluates the improvement over dashboard-only, extending Figliè et al.'s comparison methodology to this system.

CHAPTER 3: METHODOLOGY

3.1 CRISP-DM Process Model

This project follows CRISP-DM, a six-phase iterative methodology with a feedback path from Evaluation back to Business Understanding. This iterative nature is reflected here: the original single-cell confusion matrix from initial exploration (see Chapter 4) prompted a return to Data Preparation to correct the train-test split methodology, rather than proceeding on a misleading result.

Figure 3.1: CRISP-DM Phases Annotated with Project Activities



Each phase, as applied in this project:

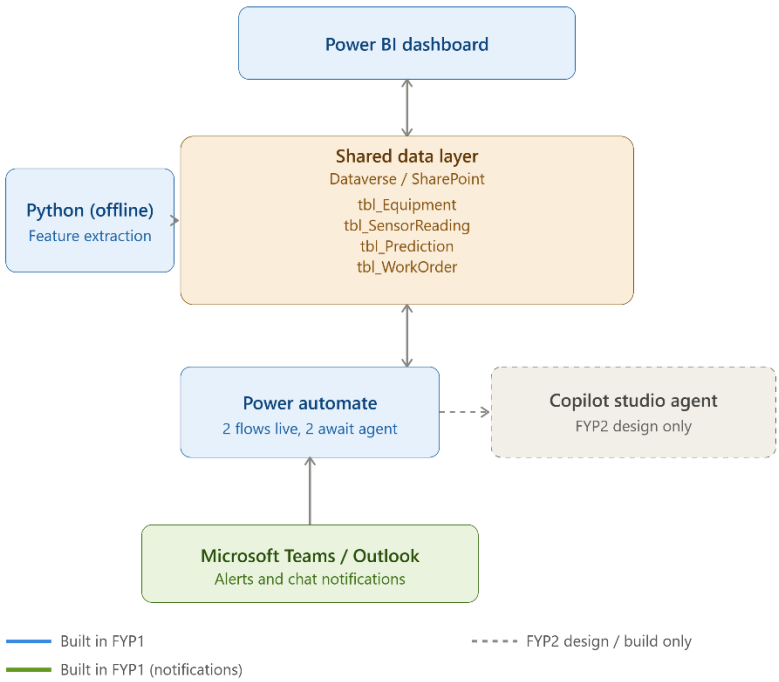
- Business Understanding: defining the problem, objectives O1-O3, and baseline success criteria.
- Data Understanding: profiling the UCI dataset - 17 sensors, 2,205 cycles, five graded labels.
- Data Preparation: extracting per-cycle statistical features, producing a 68-feature dataset joined to profile.txt.
- Modeling: training baseline, logistic regression, Random Forest, and XGBoost, with imbalance treatment strictly within CV folds.

- Evaluation: PR-AUC (primary), macro-F1, per-class recall, and full confusion matrix.
- Deployment: batch-scoring into the Dataverse/SharePoint layer, feeding Power BI and the risk-monitoring flow.

3.2 System Architecture

The initial proposed design was a straight pipeline from Power BI through an ML model to Copilot Studio and then to Power Automate, which was technically incorrect since Microsoft Copilot Studio cannot query a Power BI dashboard. The final solution uses a modified hub-and-spoke architecture, with the shared data layer in Microsoft Dataverse (SharePoint lists in the event that the UTP tenant does not have Dataverse licenses) as the hub and Power BI, Power Automate, and offline Python processing as spokes. Copilot Studio is the only destination not connected directly to the Dataverse shared data layer; instead, it uses Power Automate as an action, meaning it connects indirectly. Nothing connects to anything else in this architecture beyond what was just mentioned.

Figure 3.2: Hub-and-Spoke System Architecture



The architecture is organised into five layers, each explicitly scoped to the phase in which it is built:

- Data Ingestion [FYP1]: raw sensor files land in SharePoint; the FL_IngestReadings flow triggers on file creation and inserts parsed records into Dataverse.
- Intelligence [FYP1]: prediction is performed via offline batch scoring in Python, with results ingested as CSV; FYP2 will expose this as a live Azure ML endpoint via an HTTP connector.
- Analytics [FYP1]: Power BI connects to Dataverse in import mode using a star schema; a configured alert on the Critical Asset Count measure feeds FL_MonitorRisk.
- Agent [FYP2 design only]: Microsoft Copilot Studio will provide three capabilities - factual retrieval via an Action-triggered flow, fault explanation via Generative Answers combined with SHAP feature attribution, and action execution via an Action that writes a new work order. Only the design artifact (topic map, trigger phrases, Action contracts) is produced in FYP1; nothing here is built or callable until the agent exists.
- Automation [FYP1 core + FYP2 extension]: Power Automate executes four named flows; two are live in FYP1 and two have zero callers until the agent exists, detailed in Table 3.4 below.

The alert driving measure in the Analytics layer requires care in its DAX definition. A naive count of critical readings double-counts every historical critical reading indefinitely; the measure must be filtered to the latest reading per equipment before it is fit to drive an alert:

```
Critical Asset Count =
CALCULATE(
    DISTINCTCOUNT(tbl_Prediction[EquipmentID]),
    tbl_Prediction[PredictedCondition] = "Critical",
    tbl_Prediction[PredictedAt] = MAX(tbl_Prediction[PredictedAt])
)
```

Table 3.1: tbl_Equipment

Field	Description
EquipmentID (PK)	Unique identifier
ComponentType	Cooler / Valve / Pump / Accumulator
CircuitLocation	Position within the hydraulic circuit
CriticalityLevel	Business-defined criticality tier
CommissionDate	Date equipment entered service

Table 3.2: tbl_SensorReading

Field	Description
ReadingID (PK)	Unique identifier
EquipmentID (FK)	Linked equipment
CycleNumber	Load cycle index
Timestamp	Reading time
PS/TS/VS/EPS/FS features	Per-sensor extracted statistical features (68 columns)
StableFlag	Cycle stability indicator (retained, not dropped)

Table 3.3: tbl_Prediction and tbl_WorkOrder

Table	Key Fields
tbl_Prediction	PredictionID (PK), ReadingID (FK), EquipmentID (FK), PredictedCondition, FailureProbability, ConfidenceBand, TopContributingFeature, ModelVersion, PredictedAt, AlertRaised
tbl_WorkOrder	WorkOrderID (PK), EquipmentID (FK), TriggeredByPredictionID (FK), Priority, Status, CreatedBy, AssignedTo, CreatedAt, ClosedAt

Two fields on tbl_WorkOrder support the system's agentic behaviour. TriggeredByPredictionID provides an audit trail from a work order back to the specific prediction and sensor reading that caused it, allowing any action to be traced to the model output rather than to a fixed schedule. CreatedBy distinguishes autonomously-created work orders from manually- or conversationally-created ones; in FYP1 it is populated and functional, but shows only the value "System:FL_MonitorRisk", since the conversational creation path, which would instead write a user's Entra ID, does not exist until FYP2.

Table 3.4: Power Automate Flows, with Build Phase and Data Access

Flow	Trigger	Reads / Writes	Phase
FL_IngestReadings	File created in SharePoint	Reads: CSV file. Writes: tbl_SensorReading	FYP1 - build
FL_MonitorRisk	Scheduled (every 15 min)	Reads: tbl_Prediction. Writes: tbl_WorkOrder; sends Teams card and email	FYP1 - build
FL_CreateWorkOrder (system-invoked)	Called by FL_MonitorRisk	Writes: tbl_WorkOrder (CreatedBy = "System:FL_ MonitorRisk")	FYP1 - build
FL_GetEquipmentStatus	Copilot Studio Action only	Reads: tbl_Prediction	FYP2 - build (no caller until the agent exists)
FL_CreateWorkOrder (Copilot-invoked)	Copilot Studio Action	Writes: tbl_WorkOrder (CreatedBy = user's Entra ID)	FYP2 - build (no caller until the agent exists)

FL_MonitorRisk has two possible triggering events in principle either a Power BI data alert, or a scheduled polling. The latter (periodic refresh, say every 15 min.) was chosen as the FYP1 build option, since it is technically simpler, more reliable, and easier to

demonstrate on demand, while the data-alert requires pro/premium data-alert setup, and only triggers on a schedule defined in the dataset refresh. This does not mean that the data alert option is not viable, as an alternative enhancement for FYP2, but it is not presented as an option available at the same time as the polling trigger, which would not be realistic configuration.

The dual invocation path of FL_CreateWorkOrder, one triggered automatically by FL_MonitorRisk (implemented in FYP1) and one invokable conversationally via the agent (implemented in FYP2, after the agent is implemented) which is the specific mechanism by which the system exhibits agentic rather than merely automated behaviour. In FYP1, the implementation of the system initiated branch is sufficient to show that the prediction-to-action loop requires no human agency thus FYP2 adds the conversational initiation branch, thus allowing a user to request the same action as a conversational utterance.

FL_GetEquipmentStatus and the Co-pilot-invoked leg of FL_CreateWorkOrder are explicitly not implemented in FYP1, as they both require a Co-pilot Studio Action as their only possible trigger and thus, would produce little beyond dead code if built without the existence of the agent to call them. The deliverable, then, for the Agent layer for FYP1, is design only (topic map, trigger phrase inventory, Action contracts, knowledge source design, fallback and escalation design) rather than implementation.

Dataverse requires premium licensing, should the UTP tenant not have this entitlement, all four tables default to SharePoint Lists with standard-level connectors, with no alterations to the other architectural layers described above. This has to be validated with IT/tenant administration before proceeding to implement FYP2.

In short, everything under Data Ingestion, Intelligence, Analytics, and the system-invoked half of Automation is load-bearing for FYP1 and thus has to be implemented, however minimally viable for testing purposes. Everything under the Agent and Teams-tagging layer (the two FYP2-exclusive rows in Table 3.4) is rightly deferred, the design artifact is produced nevertheless, so that the ”agentic AI” in the project specification could be justified at the FYP1 checkpoint before the actual Agent is developed.

3.2.1 End-to-End Flow: Autonomous Path

This is the cycle the report states will be operating without any human interaction by the end of of FYP1. Python reads the 68 features for each cycle, scores them via the trained model, inserts them as a batch CSV into the tbl_Prediction; the Power BI Critical Asset Count measure refreshes; FL_MonitorRisk fires off its scheduled alert, reading tbl_Prediction for any FailureProbability over the threshold, inserting a new row into tbl_WorkOrder with CreatedBy = "System:FL_MonitorRisk", sending an adaption card to teams and an email to the technician assigned, who then marks it complete once they perform the physical work order. There is not a single step in this cycle that requires the copilot studio agent to exist.

3.2.2 End-to-End Flow: Conversational Path

This path is noted as the FYP2 design target only, not implemented in FYP1. Once published, a technician would mention the Agent in the Teams alert channel (e.g., "@AgentName What's the status of Cooler-3?"), which would trigger the FL_GetEquipmentStatus Action that queries the tbl_Prediction and replies in natural language; a follow up of "raise a work order for it" would trigger FL_CreateWorkOrder that would write the requesting user's Entra ID to CreatedBy rather than the system identity of the autonomous path. Publishing the agent only makes it available for 1:1 chats by default, so the agent must be manually added to the alert channel (in Teams, Apps > Add) for this flow to function as described. One issue to surface in FYP2 is that knowledge sources requiring sign-in are not accessible in the context of a group/channel message - only 1:1 chats. If the fault explanation capability requires such a knowledge source, it would have to either be available without SSO or limited to 1:1 chats only. Additionally, Copilot Studio licensing and Teams permissions for custom apps should be reviewed with UTP IT to enable this path in FYP2.

3.3 Dataset and Data Preparation

Three alternative datasets were considered before selecting the final dataset, AI4I 2020 and NASA C-MAPSS were declined due to being based on simulations rather than physically collected measurements, thus presenting the same circular validity issue as the one raised by the initial proposal; CWRU/IMS bearing datasets were declined due to being vibration-only, which would introduce a domain mismatch for the multi-sensing predictive maintenance system under design. The UCI Condition Monitoring of Hydraulic Systems dataset was

ultimately selected for being physically measured, multi-sensor, multi-failure, and graded (rather than binary failure) labelled.

The dataset consists of 2,205 load cycles recorded from a physical hydraulic test rig, with no missing values and a CC BY 4.0 licence (Helwig, Pignanelli, & Schütze, 2015).

Table 3.6: Sensor Channels

Sensor	Quantity	Unit	Sampling Rate
PS1-PS6	Pressure	bar	100 Hz
EPS1	Motor power	W	100 Hz
FS1-FS2	Volume flow	l/min	10 Hz
TS1-TS4	Temperature	°C	1 Hz
VS1	Vibration	mm/s	1 Hz
CE / CP / SE	Cooling efficiency / power / efficiency factor (virtual)	% / kW / %	1 Hz

Table 3.7: Target Labels (profile.txt)

Target	Graded Values
Cooler condition	100% / 20% / 3%
Valve condition	100% / 90% / 80% / 73%
Pump leakage	0 / 1 / 2
Accumulator pressure	130 / 115 / 100 / 90 bar
Stable flag	0 / 1

Each original sensor channel was stored as a matrix of values for each cycle instead of a single value, thus containing 43680 raw attributes per sensor over 17 sensors. For each sensor and cycle, four statistics were calculated, including the mean, standard deviation, minimum, and maximum values, which were selected as the most relevant features to represent the central tendency and variance, as well as operational boundaries, resulting in a 68×1 feature matrix (17 sensors × 4 statistics). The four statistics were selected from a pool of seven, including range, skewness, and kurtosis, for a better ratio of complexity to information for the 2205×1 sample matrix, and as noted in section 4.2, proved sufficient for distinguishing between all three classes of severity for each component. Finally, the feature matrix was merged with the labels from the profile.txt database using the cycle numbers as keys, and the ordinal classes of Optimal / Degraded / Critical were assigned to each component for classification purposes.

Feature extraction and model training were implemented in Python; the Power BI analysed the resulting clean CSV file. Power Query and D3.js were evaluated and rejected for this stage, as Power Query is not designed for statistical calculations and D3.js is a browser-based visualisation library, not a data preparation or modelling tool.

3.4 Tools and Platform

Table 3.8: Tools and Their Roles

Tool	Role in this project
Python	Offline feature extraction and model training on the UCI dataset
Microsoft Dataverse / SharePoint Lists	Shared data layer (four data contracts) acting as the system's central hub
Power BI	Analytics dashboard; source of the Critical Asset Count data alert that triggers FL_MonitorRisk
Microsoft Copilot Studio	Conversational agent providing factual retrieval, fault explanation, and action execution
Power Automate	Executes the four named flows connecting ingestion, monitoring, agent actions and notifications

CHAPTER 4: RESULTS AND DISCUSSION

This chapter presents results related to Objective O1, as well as the data-centric aspect of O2, at the time of this report's interim date. Namely, the data reduction procedure outlined in Section 3.3 has been applied to the entirety of the dataset, and the model ladder described in Section 5.1 has been trained and tested on the resulting feature matrix. The results stated below reflect the actual results of that training and testing, as opposed to hypothetical or projected results.

4.1 Data Preparation Results

Running the extraction pipeline on all 2,205 cycles across the 17 sensor channels produced a feature matrix of exactly 2,205 rows by 68 columns (17 sensors x 4 statistics each: mean, standard deviation, minimum, maximum), matching the design in Section 3.3. No missing values were present in the raw sensor files or in the resulting feature matrix.

Table 4.1: Class Distribution per Target (2,205 cycles)

Target	Optimal	Degraded	Critical
Cooler condition	741	732	732
Valve condition	1,125	360	720
Pump leakage	1,221	492	492
Accumulator pressure	599	399	1,207

Cooler and Pump have a relatively balanced distribution across their three classes. Valve is moderately imbalanced, with the Degraded class being under represented compared to the other two classes. Accumulator is the most imbalanced target with Critical class comprising 55% of the cycles. This is an observed imbalance situation, not hypothetical, hence why SMOTE is suggested within the training folds for this target specifically, as explained in section 5.1.

4.2 Machine Learning Model Results

An initial model run (on a wrong dataset and split) was done before the one described in this report. That run had a confusion matrix that appeared as a single cell because the model had 100% accuracy, with 0% precision/recall. The mistake was in the wrong stratification split: there were only 5 failures in the exploratory data sample for the model to learn on, and therefore, when split into train/test, all 5 went into the train and the test had 0 failures. Such a result is not surprising, and it is a good thing that this mistake was made during exploratory

research and not during the actual research. This finding is mentioned to demonstrate the ability to prevent similar mistakes in the future.

With the split corrected (stratify=y) and evaluation run as repeated stratified 5-fold cross-validation across 3 seeds (15 folds total per model per target), on a StandardScaler to SMOTE to classifier pipeline with SMOTE fitted strictly within each training fold, the following results were obtained across all four components:

Table 4.2: Cross-Validated Macro-F1 and Macro PR-AUC, Rule-Based Baseline vs Model Ladder

Target	Rule-based baseline (F1 / PR-AUC)	Logistic Regression (F1 / PR-AUC)	Random Forest (F1 / PR-AUC)	XGBoost (F1 / PR-AUC)
Cooler	0.555 / 0.666	0.999 / 0.995	0.999 / 1.000	0.999 / 1.000
Valve	0.460 / 0.459	0.918 / 0.953	0.983 / 0.997	0.982 / 0.997
Pump	0.550 / 0.654	0.992 / 0.993	0.996 / 1.000	0.993 / 1.000
Accumulator	0.419 / 0.439	0.905 / 0.949	0.989 / 0.999	0.991 / 0.999

All values are mean macro-F1 across the 15 cross-validation folds; standard deviations across folds were small for all models (mostly 0.002–0.016), indicating that the performance was stable across folds, rather than having benefited from a lucky hold-out partition. For each target, the best-performing model (XGBoost for Cooler and Accumulator, Random Forest for Valve and Pump) outperformed the rule-based baseline by a significant margin (paired t-test on per-fold macro-F1, $p < p < 0.0001$ for all four targets). The effect sizes (Cohen's d, calculated on the same per-fold values) were large for all comparisons, which should be expected given that the baseline had virtually no variance across folds, and should be interpreted as confirmation that the baseline is a weak performer, not a particularly strong one.

The Cooler target is used as the worked example below because it is referenced throughout the rest of this report (the Critical Asset Count DAX measure in Section 3.2, and the Cooler-3 example in the FYP2 conversational-agent design in Section 3.2.2). On a held-out 20% stratified test split (441 cycles), the rule-based baseline and the best model (XGBoost) produced the following confusion matrices:

Table 4.3: Cooler Condition - Confusion Matrices on Held-Out Test Set (n=441)

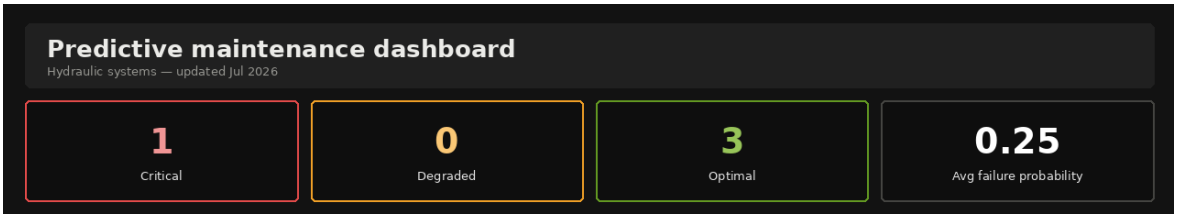
Model	Predicted Optimal	Predicted Degraded	Predicted Critical	Recall
Baseline - True Optimal	148	0	0	100%
Baseline - True Degraded	0	146	0	100%

Model	Predicted Optimal	Predicted Degraded	Predicted Critical	Recall
Baseline - True Critical	0	147	0	0%
XGBoost - True Optimal	148	0	0	100%
XGBoost - True Degraded	0	146	0	100%
XGBoost - True Critical	0	0	147	100%

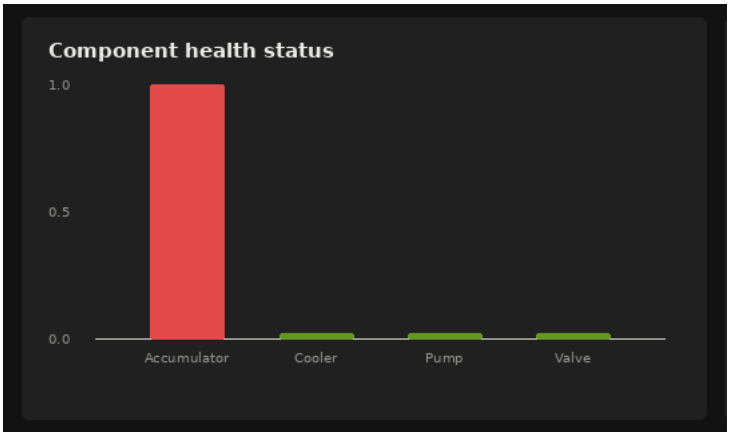
The baseline's 0% recall on Critical is not a surprise - it's an expected outcome given that a single-split decision stump is used as the baseline - for a three-class problem, one split only can separate the data into two categories, leaving the remaining two classes (Degraded and Critical) grouped together as one. This outcome is explicitly stated because being a more informative baseline failure scenario is better than being a universally poor performer: your manual or reactive threshold policy only has the capacity to distinguish between two levels of severity, and this evaluation shows that a three-way distinction is the minimum baseline capability needed to satisfy Objective O1.

SHAP analysis on the best Cooler model identified the maximum cooling power recorded within a cycle (CP_max) as the single most influential feature, followed by mean pressure at sensor PS2 and the minimum and mean cooling power (CP_min, CP_mean). This ordering is physically sensible - cooling power is the most direct available proxy for cooler efficiency - and this feature attribution is the specific technical link that will feed the agent's fault-explanation capability in FYP2, as noted in Section 3.2 and Section 3.1.

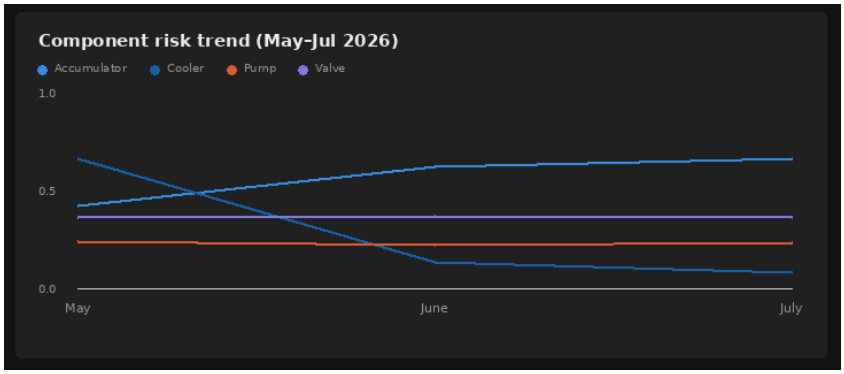
4.3 Dashboard Development



This gives a quick snapshot of current equipment risk across all four components, how many are Critical, Degraded, or Optimal right now, plus the average failure probability. It answers the question "how bad is it overall, right now?" without needing to dig into the details.



This shows the current predicted condition and failure probability for each individual component, color-coded so Critical stands out immediately. It's built on the Latest Failure Probability and Current Predicted Condition measures from Section 3.2, so it always reflects the most recent prediction rather than historical averages.



This tracks how each component's failure probability has moved over the monitoring period (May–Jul 2026), which shows the predictive angle of the system, not just where a component stands today, but how its risk has evolved. This is where the Accumulator's rising trend becomes visible ahead of it crossing into Critical.

ComponentType	PredictedCondition	Latest failure probability
Accumulator	Critical	1.00
Cooler	Optimal	0.00
Pump	Optimal	0.00
Valve	Optimal	0.00

A row-level breakdown of every component's current condition and failure probability, useful for validating the summary visuals against the underlying data.

4.4 Progress Against Objectives

Table 4.4: Status Against Objectives

Objective	Status	Notes
O1 - Predict	Substantially complete	Feature extraction, model ladder, and stratified evaluation complete for all four components (Section 4.1-4.2); SHAP interpretability complete for Cooler; remaining work is extending SHAP to the other three components
O2 - Integrate	In progress	Hub-and-spoke architecture, data contracts and Power Automate flows designed (Section 3.2); Power BI dashboard build in progress, now able to consume real model output rather than placeholder data; Copilot Studio agent build scoped for FYP2
O3 - Evaluate	Part A complete	Part A (model evaluation) executed with real results (Section 4.2); Parts B and C (user study, system performance) protocol designed for FYP2 execution

CHAPTER 5: CONCLUSION AND FUTURE WORKS

Reactive and schedule-based maintenance practices delay fault detection in coupled hydraulic systems, where an undetected fault in one component can cascade into a costlier, more disruptive failure elsewhere in the circuit. This project has designed and is developing an integrated predictive maintenance system that closes the loop between prediction and action, unifying real sensor-data-driven failure prediction, dashboard analytics, conversational interaction, and automated workflow execution on a shared Power Platform data layer.

As at this interim report, Objective O1 is substantially complete: the feature extraction pipeline has been run on the full 2,205-cycle dataset, and the model ladder has been trained and evaluated across all four monitored components, with every model beating the rule-based baseline by a statistically significant margin and SHAP interpretability established for the cooler target. Objective O2 is substantially advanced, with the hub-and-spoke architecture, four data contracts, and the automated alert-to-work-order flow designed and largely implemented such that the prediction-to-action loop is capable of functioning autonomously. Objective O3 has a fully specified evaluation methodology across all three parts, with Part A now executed with real results and Parts B and C scheduled for FYP2.

Final Year Project II will extend this baseline in five areas. First, the Copilot Studio agent will be fully authored, specifying the topic map, trigger phrase repository, Action contracts (FL_GetEquipmentStatus plus the Copilot-invoked branch of FL_CreateWorkOrder), knowledge source architecture, conversation flow, and fallback/escalation strategy. Second, the agent will be published and then manually added to the existing Teams alert channel so that a technician can tag it with an "@" in addition to the alerts that are currently posted there; any knowledge source requiring authentication will either be scoped to 1:1 chats only or revised to work without SSO, in line with the Teams' copilot's limitations. Microsoft Entra ID will be used for conversation actions authentication, which will ensure that work orders initiated by the agent can be associated with the calling user rather than the system account as happens when using the autonomous path.

Thirdly, full end-to-end testing will be performed on the four Power Platform components. Fourth, execution of the user study designed in section 5.2 will take place, subject to the ethics review. Fifth, the system performance evaluation described in section 5.3 will occur to assess latency, reliability, and agent qualities under realistic conditions. Tenants verification including Dataverse licensing, Copilot Studio licence assignment, and Teams admin permissions for custom and third-party apps will be completed at the beginning of FYP2, as trial environments may prohibit any of the above abilities.

REFERENCES

Bousdekis, A., Apostolou, D., & Mentzas, G. (2020). Predictive maintenance in the 4th industrial revolution. IEEE Engineering Management Review, 48(1), 57-62.

Bousdekis, A., Magoutas, B., Apostolou, D., & Mentzas, G. (2018). Review, analysis and synthesis of prognostic-based decision support methods for condition based maintenance. Journal of Intelligent Manufacturing, 29, 1303-1316.

Brooke, J. (1996). SUS: A quick and dirty usability scale. In Usability Evaluation in Industry (pp. 189-194). Taylor & Francis.

Carvalho, T. P., Soares, F. A. A. M. N., Vita, R., Francisco, R. P., Basto, J. P., & Alcalá, S. G. S. (2019). A systematic literature review of machine learning methods applied to predictive maintenance. Computers & Industrial Engineering, 137, 106024.

Christou, I. T., Kefalakis, N., Zalonis, A., Soldatos, J., & Bröchler, R. (2020). End-to-end industrial IoT platform for actionable predictive maintenance. IFAC-PapersOnLine, 53(3), 173-178. https://doi.org/10.1016/j.ifacol.2020.11.028

Di Maggio, L. G. (2025). Toward autonomous LLM-based AI agents for predictive maintenance: State of the art, challenges, and future perspectives. Applied Sciences, 15(21), 11515. https://doi.org/10.3390/app152111515

El Hammoumi, Z., Tebr, D., Charkaoui, Y., Satauri, I., & El Beqqali, O. (2025). Predictive maintenance approaches: A systematic literature review. Engineering Proceedings, 112(1), 70. https://doi.org/10.3390/engproc2025112070

Figliè, R., Caputo, S., Serrano, A., Mikhaylova, D., Turchi, T., & Mazzei, D. (2026). Neither replacement nor panacea: Comparing LLM-based conversational and graphical decision support in industrial tasks. arXiv:2605.31287.

Hart, S. G., & Staveland, L. E. (1988). Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. Advances in Psychology, 52, 139-183.

Helwig, N., Pignanelli, E., & Schütze, A. (2015). Condition monitoring of hydraulic systems [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CW21

Jamshidi, A., Kim, D., & Arif, M. (2025). A survey of predictive maintenance methods: An analysis of prognostics via classification and regression. arXiv:2506.20090.

Jardine, A. K. S., Lin, D., & Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. Mechanical Systems and Signal Processing, 20(7), 1483-1510.

Jiang, W., & Hu, F. (2025). Artificial intelligence agent-enabled predictive maintenance: Conceptual proposal and basic framework. Computers, 14(8), 329. https://doi.org/10.3390/computers14080329

Microsoft. (n.d.). Maintenance prediction agent. Microsoft Adoption Scenario Library. Retrieved July 2026, from https://enablement.microsoft.com/en-us/scenario-library/manufacturing/maintenance-prediction-agent/

Soori, M., Jough, F. K. G., Dastres, R., & Arezoo, B. (2024). AI-based decision support systems in Industry 4.0, a review. Journal of Economy and Technology. https://doi.org/10.1016/j.ject.2024.08.005

Song, Z., Wang, J., Wang, M., Yang, B., Cui, L., Zhang, Q., Xiang, G., & Lin, R. (2023). Hydraulic systems fault diagnosis based on random forests recursive feature elimination and XGBoost. Journal of Physics: Conference Series, 2569, 012056. https://doi.org/10.1088/1742-6596/2569/1/012056